



COMPREHENSIVE GUIDE

FFUF

Table of Contents

Abstract	4
Setup	5
Input Options	7
1. Simple Attack	7
2. Multiple Wordlists	8
3. Ignore Wordlist Comment and Silent	8
4. Extensions	10
5. Request Request-Proto Mode	11
Match Options	13
1. Match HTTP Code	13
2. Match Lines	14
3. Match Words	15
4. Match Size	16
5. Match Regular Expression	17
Filter Options	18
1. Filter Code	18
2. Filter Lines	19
3. Filter Size	20
4. Filter Words	21
5. Filter Regular Expression	22
General Options	23
1. Custom Auto Calibration	23
2. Color	24
3. Maxtime For Task	25
4. Maxtime For Job	26
5. Delay	27
6. Request Rate	28
7. Error Functions	29
8. Verbose Mode	30
9. Threads	31
Output Options	32
1. Output Format in HTML	32
2. Output Format in CSV	34

3. All Output Format:	36
HTTP Options	37
1. Timeout:	37
2. Host Header	38
3. Recursion	39
4. Attack with Cookie	40
5. Replay-Proxy	41
Conclusion	43
References	43
About Us	44

Abstract

We will learn how we can use ffuf, which states for “Fuzz Faster U Fool”, which is an interesting open-source web fuzzing tool. Since its release, many people have gravitated towards ffuf, particularly in the bug bounty scenario. So, let’s dive into this learning process.

It is a professional command-line method for web fuzzing on a web server and the credit goes to the author (@j00hoi). Many people have gravitated towards ffuf since its release, especially in the bug bounty scene. While the bulk of this shift is possibly attributable to the herd mentality, a significant portion of the group has made the switch due to FFUF’s tempo, versatility, and capacity to easily merge with external tooling.

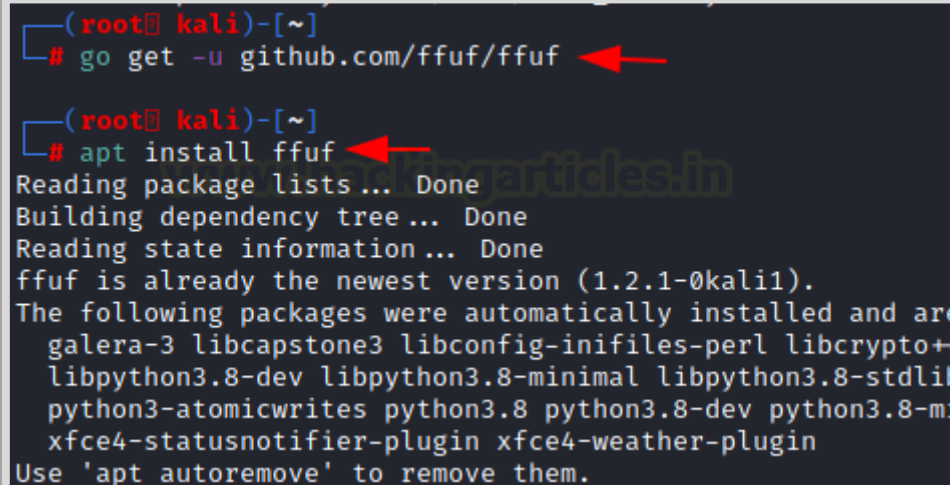
Setup

It is a command-line program that runs in the Linux Terminal or the Windows Command Prompt. Upgrading from the source is not any more difficult than compiling from the source, with the exception of the inclusion of the **-u flag**.

```
go get -u github.com/ffuf/ffuf
```

Due to the fact we are using Kali Linux, we'll find ffuf in the apt repositories, allowing us to install by running the simple command.

```
apt install ffuf
```



```
(root@kali)~# go get -u github.com/ffuf/ffuf
(root@kali)~# apt install ffuf
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ffuf is already the newest version (1.2.1-0kali1).
The following packages were automatically installed and are
  galera-3 libcapstone3 libconfig-inifiles-perl libcrypto+
  libpython3.8-dev libpython3.8-minimal libpython3.8-stdli
  python3-atomicwrites python3.8 python3.8-dev python3.8-mi
  xfce4-statusnotifier-plugin xfce4-weather-plugin
Use 'apt autoremove' to remove them.
```

After installing this tool, to get its working parameters and options all we need is just to use [-h] parameter for the help option.

```
ffuf -h
```

```

# ffuf -h
Fuzz Faster U Fool - v1.2.1

HTTP OPTIONS:
-H          Header `Name: Value`, separated by colon. Multiple -H flags are accepted
-X          HTTP method to use
-b          Cookie data `NAME1=VALUE1; NAME2=VALUE2` for copy as curl functional
-d          POST data
-ignore-body Do not fetch the response content. (default: false)
-r          Follow redirects (default: false)
-recursion  Scan recursively. Only FUZZ keyword is supported, and URL (-u) has to
-recursion-depth Maximum recursion depth. (default: 0)
-replay-proxy Replay matched requests using this proxy.
-timeout    HTTP request timeout in seconds. (default: 10)
-u          Target URL
-x          HTTP Proxy URL

GENERAL OPTIONS:
-V          Show version information. (default: false)
-ac         Automatically calibrate filtering options (default: false)
-acc        Custom auto-calibration string. Can be used multiple times. Implies -a
-c          Colorize output. (default: false)
-config     Load configuration from a file
-maxtime    Maximum running time in seconds for entire process. (default: 0)
-maxtime-job Maximum running time in seconds per job. (default: 0)
-p          Seconds of `delay` between requests, or a range of random delay. For e
-rate       Rate of requests per second (default: 0)
-s          Do not print additional information (silent mode) (default: false)
-sa         Stop on all error cases. Implies -sf and -se. (default: false)
-se         Stop on spurious errors (default: false)
-sf         Stop when > 95% of responses return 403 Forbidden (default: false)
-t          Number of concurrent threads. (default: 40)
-v          Verbose output, printing full URL and redirect location (if any) with

MATCHER OPTIONS:
-mc         Match HTTP status codes, or "all" for everything. (default: 200,204,30
-ml         Match amount of lines in response
-mr         Match regexp
-ms         Match HTTP response size
-mw         Match amount of words in response

FILTER OPTIONS:
-fc         Filter HTTP status codes from response. Comma separated list of codes
-fl         Filter by amount of lines in response. Comma separated list of line co
-fr         Filter regexp
-fs         Filter HTTP response size. Comma separated list of sizes and ranges
-fw         Filter by amount of words in response. Comma separated list of word co

INPUT OPTIONS:
-D          DirSearch wordlist compatibility mode. Used in conjunction with -e fla
-e          Comma separated list of extensions. Extends FUZZ keyword.
-ic         Ignore wordlist comments (default: false)
-input-cmd  Command producing the input. --input-num is required when using this i
-input-num  Number of inputs to test. Used in conjunction with --input-cmd. (defau
-input-shell Shell to be used for running command

```


Input Options

These are parameters that help us to provide the required data for web fuzzing over a URL with the help of a wordlist.

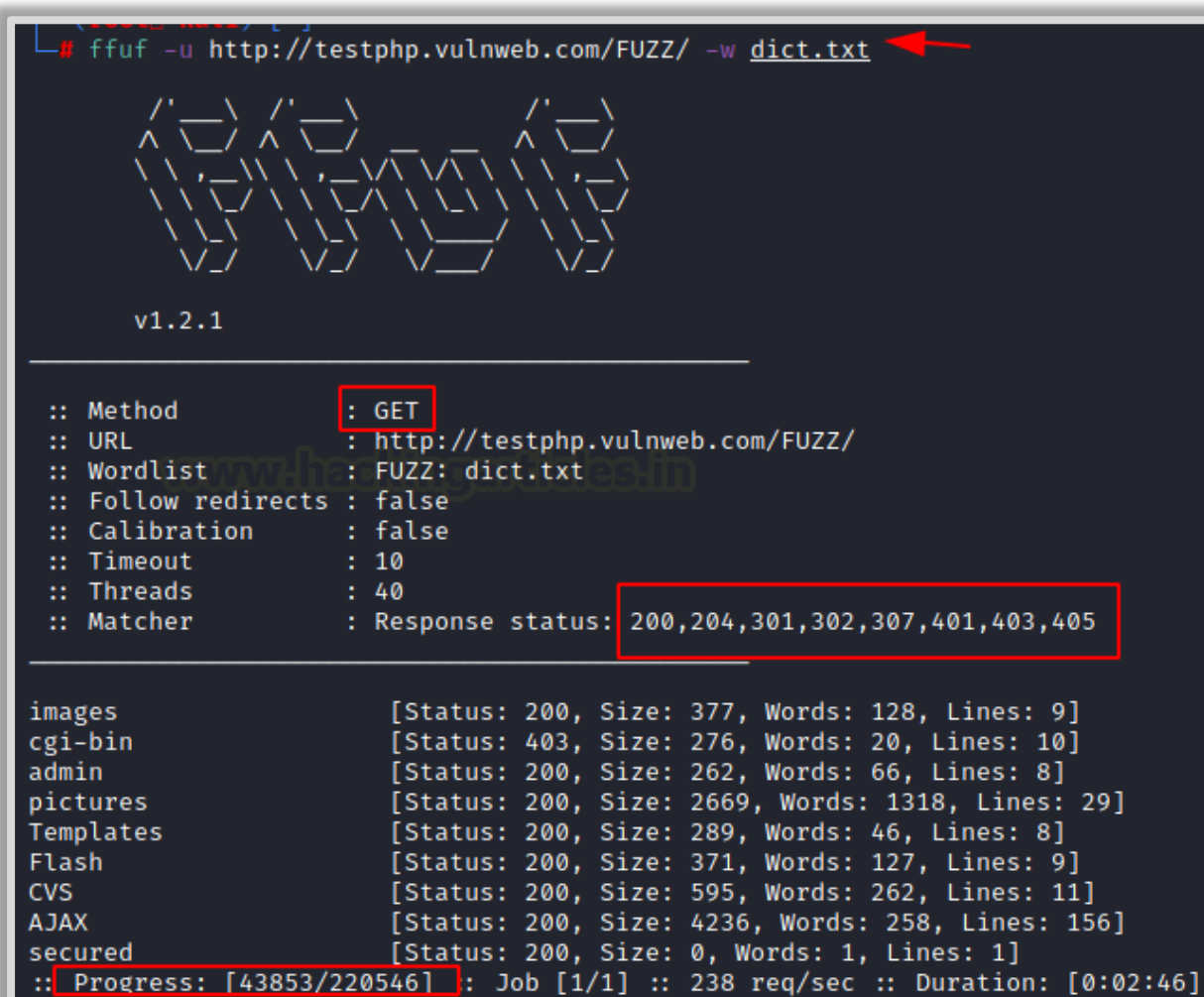
1. Simple Attack

For the default attack, we need to use parameters [-u] for the target URL and [-w] to load a wordlist as shown in the image.

```
ffuf -u http://testphp.vulnweb.com/FUZZ/ -w
```

After running the command, let's focus on the results.

- Firstly, we noticed that it is by default running on **HTTP method GET**.
- The next things are **response code status** [200, 204, 301, 302, 307, 401, 403, 405]; it also shows the progression of our attack. At the end of the progress, we got our results.



```
# ffuf -u http://testphp.vulnweb.com/FUZZ/ -w dict.txt

      ____      _
     / ___/____(_)____
    /  /__/ ___/ /   _ \
   / ____/  /  /  /  __/
  /_/____/____/_/____/

v1.2.1

:: Method      : GET
:: URL         : http://testphp.vulnweb.com/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

images      [Status: 200, Size: 377, Words: 128, Lines: 9]
cgi-bin     [Status: 403, Size: 276, Words: 20, Lines: 10]
admin       [Status: 200, Size: 262, Words: 66, Lines: 8]
pictures    [Status: 200, Size: 2669, Words: 1318, Lines: 29]
Templates   [Status: 200, Size: 289, Words: 46, Lines: 8]
Flash       [Status: 200, Size: 371, Words: 127, Lines: 9]
CVS         [Status: 200, Size: 595, Words: 262, Lines: 11]
AJAX        [Status: 200, Size: 4236, Words: 258, Lines: 156]
secured     [Status: 200, Size: 0, Words: 1, Lines: 1]
:: Progress: [43853/220546] :: Job [1/1] :: 238 req/sec :: Duration: [0:02:46]
```

2. Multiple Wordlists

Sometimes one wordlist isn't sufficient to show us our desired results. In that case, we can put multiple wordlists at once to get better results. Only ffuf has the ability to run as many wordlists as per our need for attack.

Here I provided two dictionaries dict.txt as W1 & W2 as Dns.txt and ffuf will read both dictionaries simultaneously.

```
ffuf -u https://ignitetechnologies.in/W2/W1/
-w dict.txt:W1 -w dns dict.txt:W2
```

```
(root@kali)~[~]
# ffuf -u https://hackingarticles.in/W2/W1/ -w dict.txt:W1 -w dns_dict.txt:W2

v1.2.1

:: Method      : GET
:: URL         : https://hackingarticles.in/W2/W1/
:: Wordlist    : W1: dict.txt
:: Wordlist    : W2: dns_dict.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

[Status: 301, Size: 0, Words: 1, Lines: 1]
* W1: column
* W2: mail

[Status: 301, Size: 0, Words: 1, Lines: 1]
* W1: 305
* W2: mail

[Status: 301, Size: 0, Words: 1, Lines: 1]
* W1: 211
* W2: mail
```

3. Ignore Wordlist Comment and Silent

Generally, the default wordlist might have some comments that can affect our result accuracy. In this case, we can use [-ic] parameter that can help us to get rid of that comment. Sometimes we need to be more focused on attack rather than tools banners for this kind of accuracy we need [-s] parameter which has the power to remove the banner of the tool.


```
ffuf -u http://testphp.vulnweb.com/FUZZ/ -w dict.txt
```

we can clearly see some comments are listed in the result when we have run above the command and after using [-s] & [-ic] parameters in the next command the comments and banner are removed.

```
ffuf -u http://testphp.vulnweb.com/FUZZ/ -w dict.txt -ic -s
```

```
# ffuf -u http://testphp.vulnweb.com/FUZZ/ -w dict.txt

v1.2.1

:: Method      : GET
:: URL         : http://testphp.vulnweb.com/FUZZ/
:: Wordlist     : FUZZ: dict.txt
:: Follow redirects: false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

# comment this line [Status: 200, Size: 4958, Words: 514, Lines: 110]
# Hello World      [Status: 200, Size: 4958, Words: 514, Lines: 110]
images             [Status: 200, Size: 377, Words: 128, Lines: 9]
cgi-bin            [Status: 403, Size: 276, Words: 20, Lines: 10]
admin              [Status: 200, Size: 262, Words: 66, Lines: 8]
pictures           [Status: 200, Size: 2669, Words: 1318, Lines: 29]
[WARN] Caught keyboard interrupt (Ctrl-C)

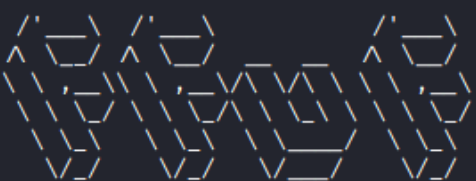
(root@kali)-[~]
# ffuf -u http://testphp.vulnweb.com/FUZZ/ -w dict.txt -ic -s
cgi-bin
images
admin
pictures
Templates
[WARN] Caught keyboard interrupt (Ctrl-C)
```

4. Extensions

We can search for a specific extension file on a web server with the help of [-e] parameter, all we need to just specify the extension file along with [-e] parameter. To get these results we just need to follow the command.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -e .php
```

```
(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -e .php
```



```
v1.2.1
```

```
:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Extensions  : .php
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
```

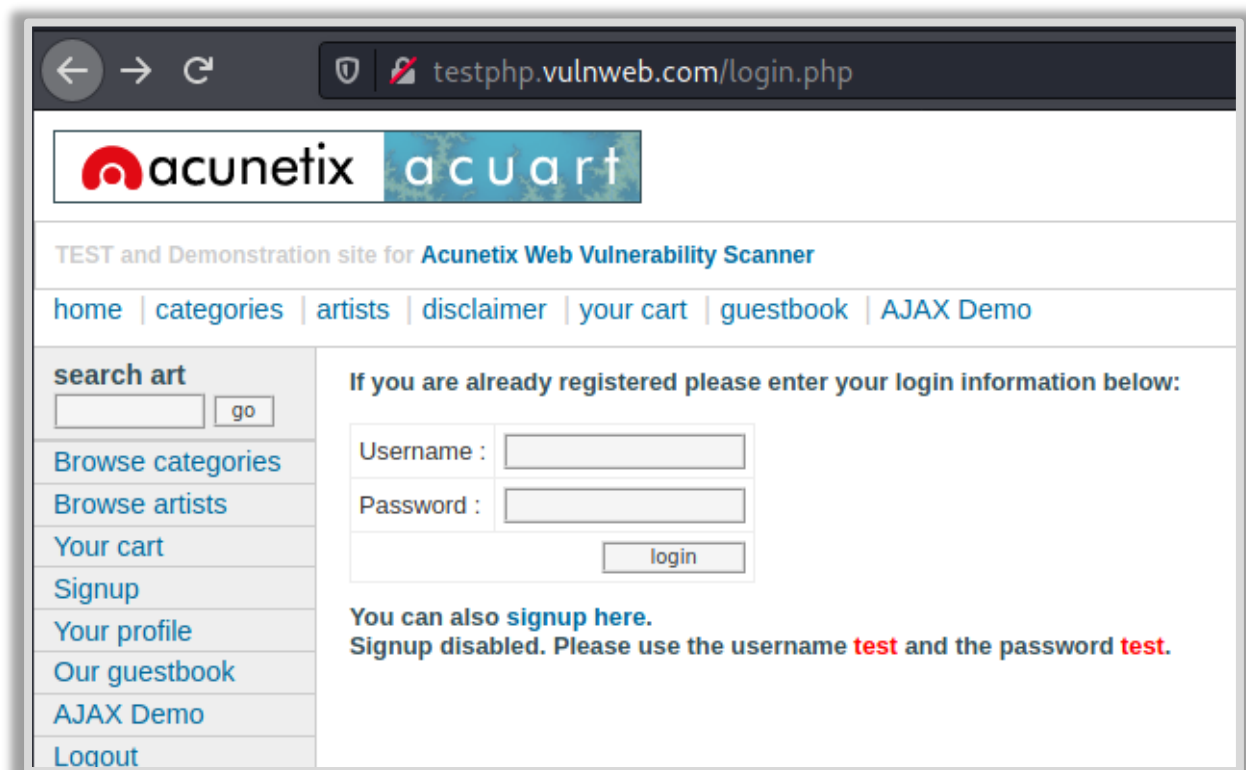
```
docs           [Status: 200, Size: 920, Words: 53, Lines: 15]
login          [Status: 200, Size: 1289, Words: 83, Lines: 66]
security.php   [Status: 302, Size: 0, Words: 1, Lines: 1]
index          [Status: 302, Size: 0, Words: 1, Lines: 1]
index.php      [Status: 302, Size: 0, Words: 1, Lines: 1]
login.php      [Status: 200, Size: 1289, Words: 83, Lines: 66]
security       [Status: 302, Size: 0, Words: 1, Lines: 1]
external       [Status: 200, Size: 898, Words: 53, Lines: 15]
about.php      [Status: 302, Size: 0, Words: 1, Lines: 1]
logout.php     [Status: 302, Size: 0, Words: 1, Lines: 1]
logout        [Status: 302, Size: 0, Words: 1, Lines: 1]
about          [Status: 302, Size: 0, Words: 1, Lines: 1]
config         [Status: 200, Size: 909, Words: 54, Lines: 15]
setup.php      [Status: 200, Size: 3549, Words: 182, Lines: 81]
setup          [Status: 200, Size: 3549, Words: 182, Lines: 81]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
instructions   [Status: 302, Size: 0, Words: 1, Lines: 1]
instructions.php [Status: 302, Size: 0, Words: 1, Lines: 1]
               [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo        [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo.php    [Status: 302, Size: 0, Words: 1, Lines: 1]
[WARN] Caught keyboard interrupt (Ctrl-C)
```

5. Request | Request-Proto | Mode

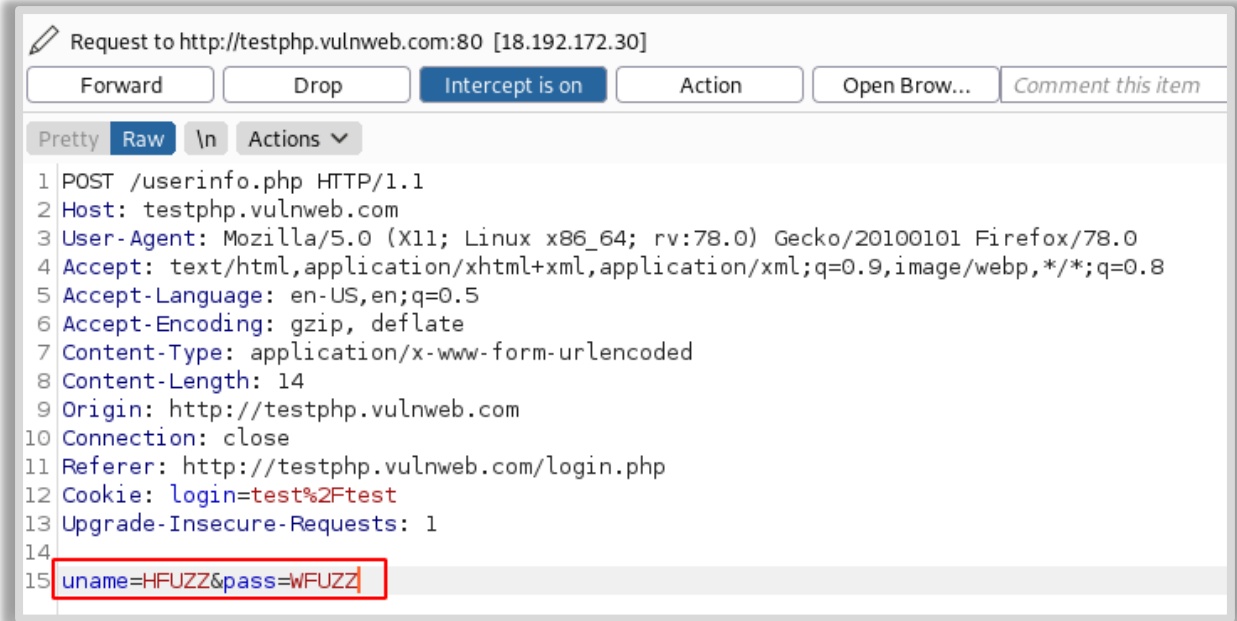
Burp Suite is an advanced framework for conducting web application security monitoring. Its different instruments act in agreement to help the testing process as a whole. A cluster bomb is a feature that uses several payload sets. For each given location, there is a different payload package. the attack goes through each payload package one by one, checking all potential payload variations.

There is a various parameter of this tool, which help to use this our scenario. Like [-request] parameter which can use our request in the attack, [-request-proto] parameter through which we can define our parameter, [-mode] parameter help us to define the mode of attack.

First of all, we use random credentials on our targeted URL page and set proxy up to capture its request in intercept mode on Burpsuite.



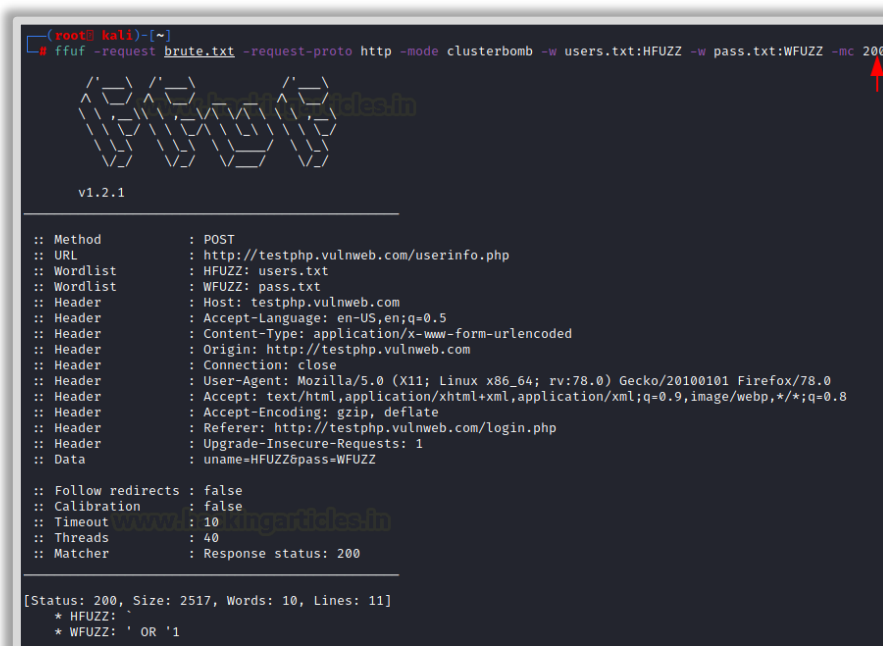
Now in the intercept tab of the Burpsuite, change our provided credential with HFUZZ and WFUZZ. Put HFUZZ in front of uname and WFUZZ in front of the pass. Then copy-paste this request in a text and name as per your desire. In our case, we named that brute.txt.



Now proceed towards the main attack, where [-request] parameter hold our request text file. [-request-proto] help us derive the http prototype [-mode] help us to derive us cluster bomb attack. The wordlists we use in these (users.txt and pass.txt) consist of SQL injections. Follow this command start attacking using these parameters.

```
ffuf -request brute.txt -request-proto http
-mode clusterbomb -w users.txt:HFUZZ -w
pass.txt:WFUZZ -mc 200
```

as we can see in our attack results, we have successfully found out SQL injections working on that particular target.



Match Options

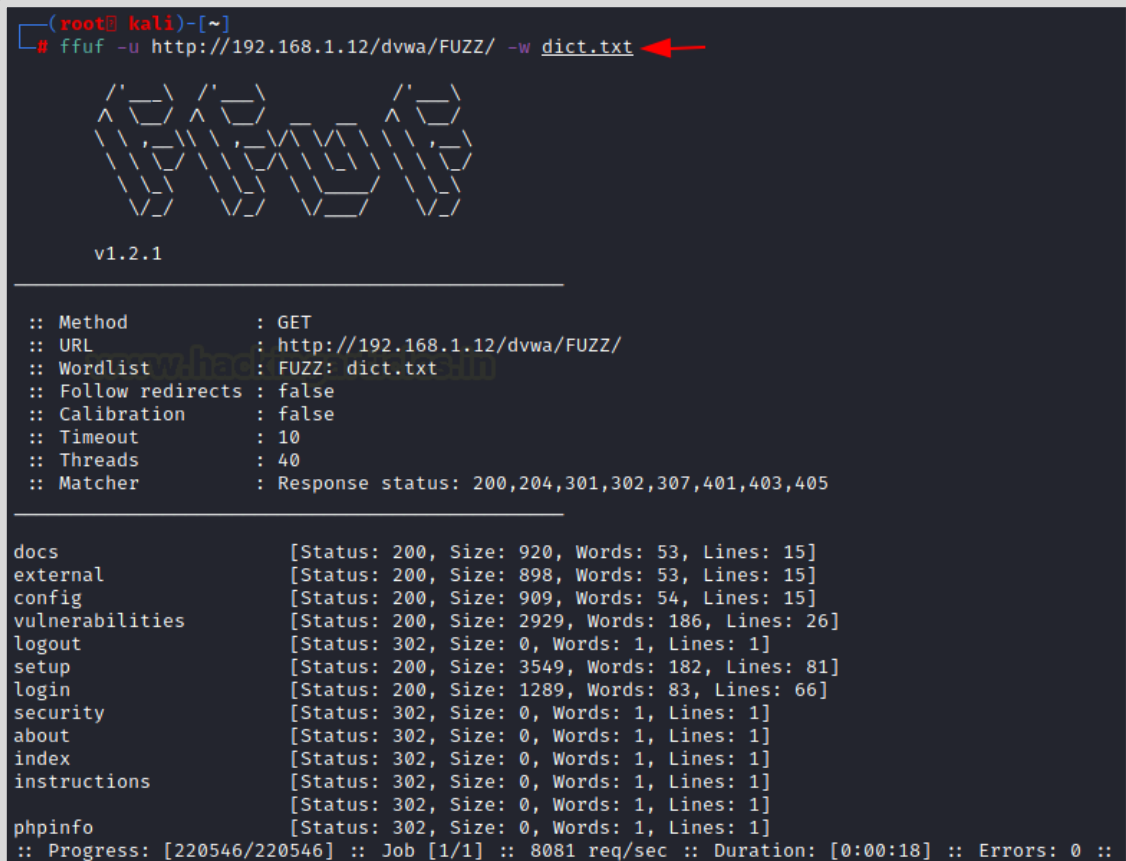
If we want ffuf to show only that data which is important in our web fuzzing data. Then it will help us to showcase only matched according to the parameter. Example: HTTP code, Lines, Words, Size and Regular Expressions.

1. Match HTTP Code

To get an understanding of this parameter we need to consider a simple attack where we can see which HTTP codes are appearing in our results.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt
```

we can clearly see that it showing some 302 HTTP code along with 200 HTTP code.



```
(root@kali)~# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt
```

```
v1.2.1
```

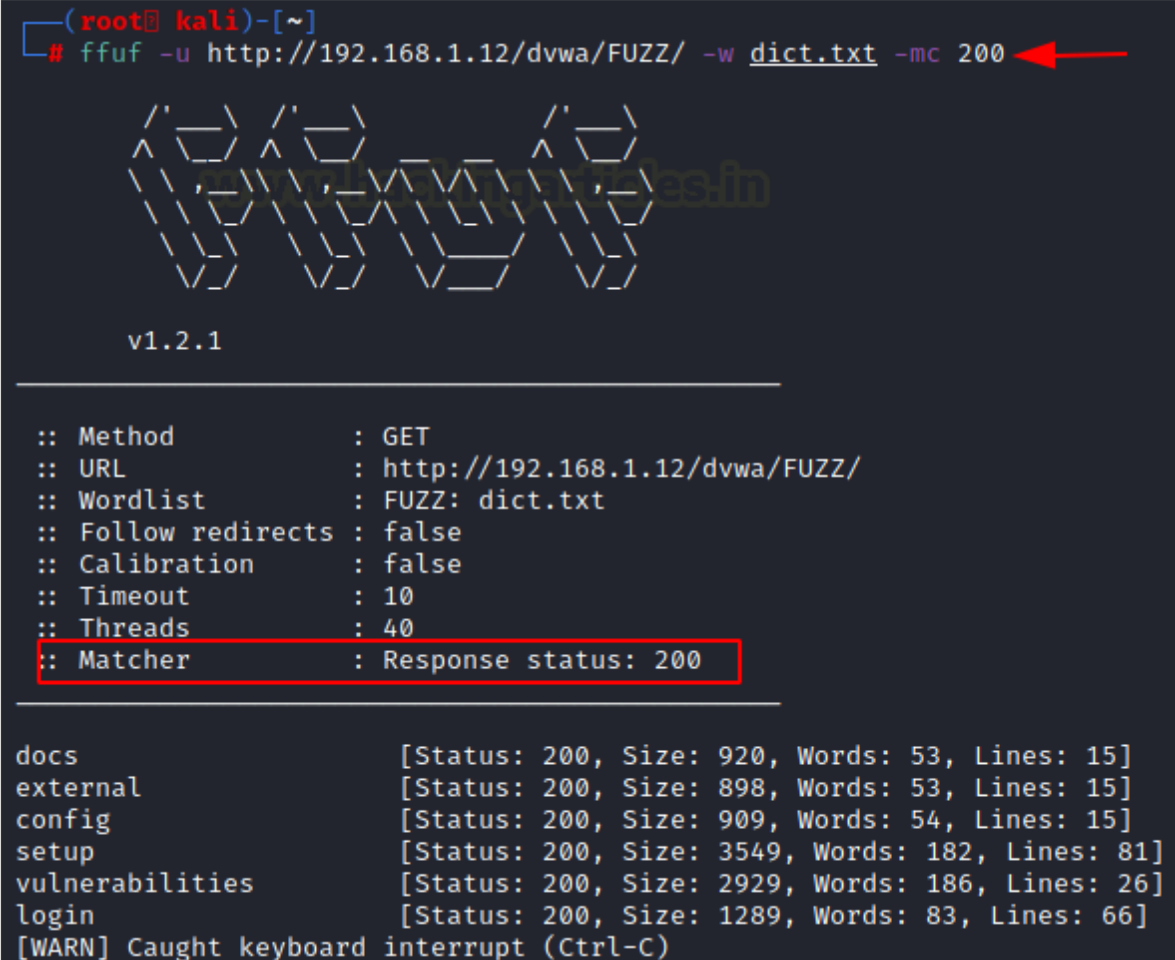
```
:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
```

```
docs      [Status: 200, Size: 920, Words: 53, Lines: 15]
external  [Status: 200, Size: 898, Words: 53, Lines: 15]
config    [Status: 200, Size: 909, Words: 54, Lines: 15]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
logout    [Status: 302, Size: 0, Words: 1, Lines: 1]
setup     [Status: 200, Size: 3549, Words: 182, Lines: 81]
login     [Status: 200, Size: 1289, Words: 83, Lines: 66]
security  [Status: 302, Size: 0, Words: 1, Lines: 1]
about     [Status: 302, Size: 0, Words: 1, Lines: 1]
index     [Status: 302, Size: 0, Words: 1, Lines: 1]
instructions [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo   [Status: 302, Size: 0, Words: 1, Lines: 1]
:: Progress: [220546/220546] :: Job [1/1] :: 8081 req/sec :: Duration: [0:00:18] :: Errors: 0 ::
```

If only need successful results like 200 HTTP code we just need to use [-mc] parameter along with our specific HTTP code. To use this parameter just follow the command.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -mc 200
```

```
(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -mc 200
```



```

v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200

docs           [Status: 200, Size: 920, Words: 53, Lines: 15]
external       [Status: 200, Size: 898, Words: 53, Lines: 15]
config         [Status: 200, Size: 909, Words: 54, Lines: 15]
setup          [Status: 200, Size: 3549, Words: 182, Lines: 81]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
login          [Status: 200, Size: 1289, Words: 83, Lines: 66]
[WARN] Caught keyboard interrupt (Ctrl-C)

```

2. Match Lines

Like the match code which we discussed earlier, it gives us the result for a specific-lines in a file with the help of [-ml] parameter. We can use this [-ml] parameter by specifying the lines we need in a file.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -ml 15
```

```
(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -ml 15

v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist     : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher      : Response lines: 15

docs          [Status: 200, Size: 920, Words: 53, Lines: 15]
external      [Status: 200, Size: 898, Words: 53, Lines: 15]
config        [Status: 200, Size: 909, Words: 54, Lines: 15]
[WARN] Caught keyboard interrupt (Ctrl-C)
```

3. Match Words

Similarly, as the above functionalities match function it can provide us with a result with a specific word count. To get this result we need to use [-mw] parameter along specific words count we want in our results.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -mw 53
```



```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -mw 53

```



 www.hackingarticles.in

v1.2.1

```

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response words: 53

```

```

docs      [Status: 200, Size: 920, Words: 53, Lines: 15]
external  [Status: 200, Size: 898, Words: 53, Lines: 15]

```

4. Match Size

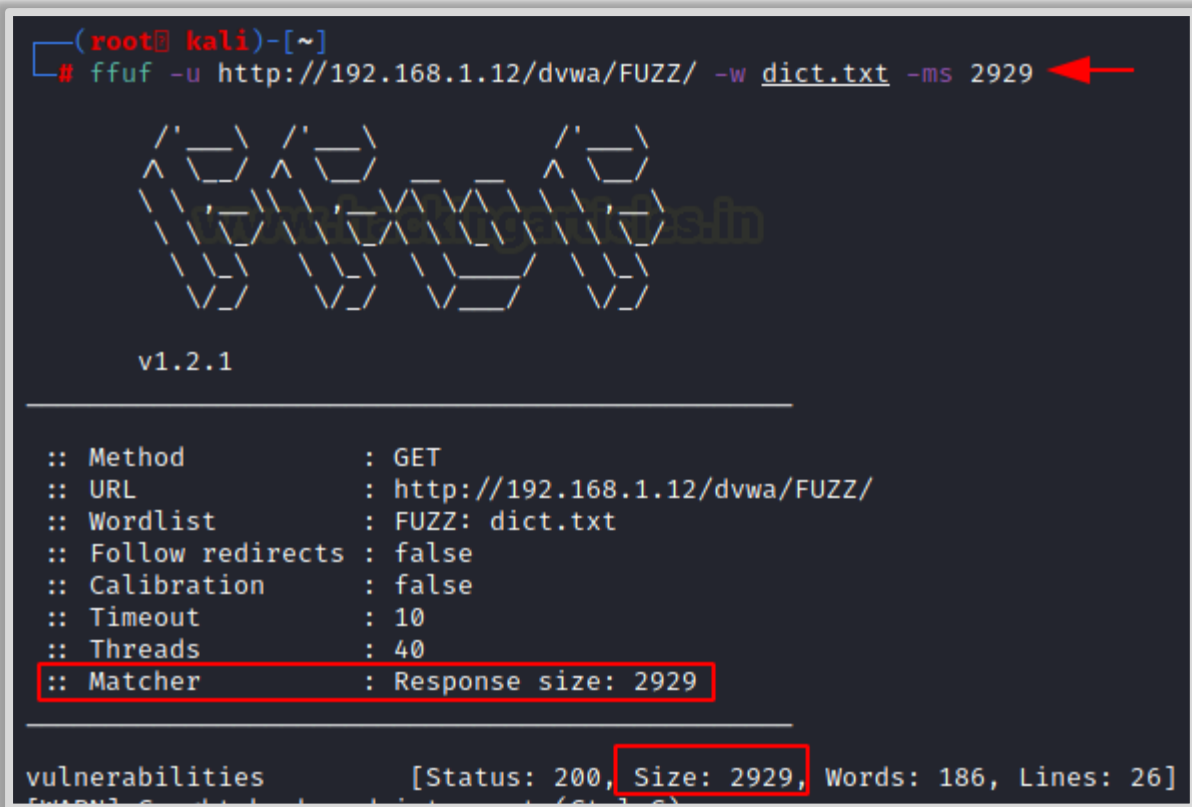
Similarly, as the above functionalities match function it can provide us with a result with the size of the file. We can use [-ms] parameter along with the specific size count we want in our result.

```

ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -ms 2929

```

```
(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -ms 2929
```



```
v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist     : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher      : Response size: 2929

vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
```

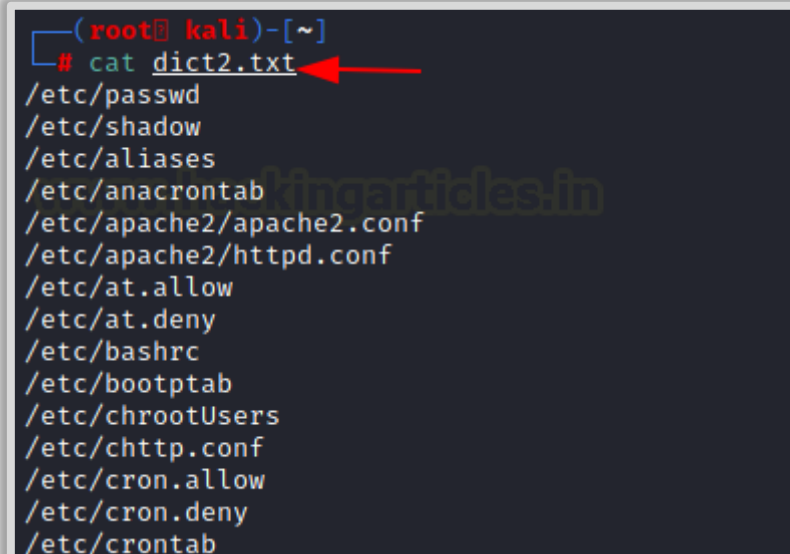
5. Match Regular Expression

It is the last of all match functions available in this tool. We are going to fuzz for LFI by matching the string with followed pattern “root:x” for the given dictionary.

We are using a URL that can achieve this functionality and by using [-mr] parameter we define the matching string “root:x”.

This our special wordlist looks like.

```
(root@kali)-[~]
# cat dict2.txt
```



```
/etc/passwd
/etc/shadow
/etc/aliases
/etc/anacrontab
/etc/apache2/apache2.conf
/etc/apache2/httpd.conf
/etc/at.allow
/etc/at.deny
/etc/bashrc
/etc/bootptab
/etc/chrootUsers
/etc/chttp.conf
/etc/cron.allow
/etc/cron.deny
/etc/crontab
```

By using this wordlist, follow the below command to use [-mr] parameter in an attack scenario.

```
ffuf -u http://testphp.vulnweb.com/showimage.php?file=FUZZ
-w dict2.txt -mr "root:x"
```

Here we got HTTP to respond 200 for /etc/passwd for the given wordlist.

```
(root@kali)~# ffuf -u http://testphp.vulnweb.com/showimage.php?file=FUZZ -w dict2.txt -mr "root:x"
v1.2.1

:: Method      : GET
:: URL         : http://testphp.vulnweb.com/showimage.php?file=FUZZ
:: Wordlist     : FUZZ: dict2.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout      : 10
:: Threads      : 40
:: Matcher      : Regexp: root:x

..../etc/passwd      [Status: 200, Size: 845, Words: 4, Lines: 24]
:: Progress: [112/112] :: Job [1/1] :: 0 req/sec :: Duration: [0:00:00] :: Errors: 0 ::
```

Filter Options

The Filter options are absolutely opposite to Match options. We can use these options to remove the unwanted from our web fuzzing. Example: HTTP Code, Lines, Words, Size, Regular Expressions.

1. Filter Code

The [-fc] parameter need the specific HTTP status code we want to remove from the result.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -fc 302
```

```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -fc 302

```

v1.2.1

```

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
:: Filter      : Response status: 302

```

```

docs           [Status: 200, Size: 920, Words: 53, Lines: 15]
external       [Status: 200, Size: 898, Words: 53, Lines: 15]
config         [Status: 200, Size: 909, Words: 54, Lines: 15]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
login          [Status: 200, Size: 1289, Words: 83, Lines: 66]
setup          [Status: 200, Size: 3549, Words: 182, Lines: 81]
:: Progress: [220546/220546] :: Job [1/1] :: 11843 req/sec :: Duration: [

```

2. Filter Lines

The [-fl] parameter has the ability to remove a specific length from our result or we can filter it out from our attack.

```

ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -fl 26

```

```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -fl 26

```

v1.2.1

```

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
:: Filter      : Response lines: 26

```

```

docs      [Status: 200, Size: 920, Words: 53, Lines: 15]
index     [Status: 302, Size: 0, Words: 1, Lines: 1]
about     [Status: 302, Size: 0, Words: 1, Lines: 1]
login     [Status: 200, Size: 1289, Words: 83, Lines: 66]
external  [Status: 200, Size: 898, Words: 53, Lines: 15]
logout    [Status: 302, Size: 0, Words: 1, Lines: 1]
config    [Status: 200, Size: 909, Words: 54, Lines: 15]
setup     [Status: 200, Size: 3549, Words: 182, Lines: 81]
security  [Status: 302, Size: 0, Words: 1, Lines: 1]
instructions [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo   [Status: 302, Size: 0, Words: 1, Lines: 1]
:: Progress: [220546/220546] :: Job [1/1] :: 11570 req/sec :: Duration:

```

3. Filter Size

The [-fs] parameter has the ability to filter out the specified size is described by us during the command of the attack.

```

ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -fs 2929

```

```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -fs 2929

```



```

v1.2.1

```

```

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
:: Filter      : Response size: 2929

```

```

docs           [Status: 200, Size: 920, Words: 53, Lines: 15]
security       [Status: 302, Size: 0, Words: 1, Lines: 1]
about          [Status: 302, Size: 0, Words: 1, Lines: 1]
external       [Status: 200, Size: 898, Words: 53, Lines: 15]
logout         [Status: 302, Size: 0, Words: 1, Lines: 1]
config         [Status: 200, Size: 909, Words: 54, Lines: 15]
index          [Status: 302, Size: 0, Words: 1, Lines: 1]
setup          [Status: 200, Size: 3549, Words: 182, Lines: 81]
login          [Status: 200, Size: 1289, Words: 83, Lines: 66]
instructions   [Status: 302, Size: 0, Words: 1, Lines: 1]
               [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo        [Status: 302, Size: 0, Words: 1, Lines: 1]
:: Progress: [220546/220546] :: Job [1/1] :: 6362 req/sec :: Duration: [0

```

4. Filter Words

The [-fw] parameter has the ability to filter out the words count from results that we want to remove.

```

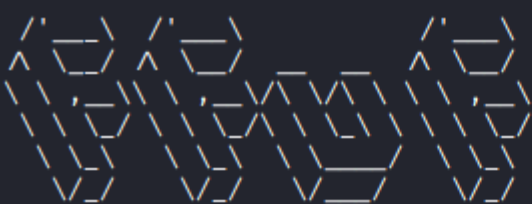
ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -fw 83

```

```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -fw 83

```



```

v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist     : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
:: Filter      : Response words: 83

index           [Status: 302, Size: 0, Words: 1, Lines: 1]
security        [Status: 302, Size: 0, Words: 1, Lines: 1]
docs            [Status: 200, Size: 920, Words: 53, Lines: 15]
external        [Status: 200, Size: 898, Words: 53, Lines: 15]
logout          [Status: 302, Size: 0, Words: 1, Lines: 1]
config          [Status: 200, Size: 909, Words: 54, Lines: 15]
setup           [Status: 200, Size: 3549, Words: 182, Lines: 81]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
about           [Status: 302, Size: 0, Words: 1, Lines: 1]
instructions    [Status: 302, Size: 0, Words: 1, Lines: 1]
                [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo         [Status: 302, Size: 0, Words: 1, Lines: 1]
:: Progress: [220546/220546] :: Job [1/1] :: 9677 req/sec :: Duration: [0

```

5. Filter Regular Expression

The parameter [-fr] we can remove a specific regular expression, here we try to exclude the log file from the output result.

```

ffuf -u http://192.168.1.12/dvwa/FUZZ/
-w dict.txt -fr log

```



```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -fr "log"

```



```

v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist     : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
:: Filter      : Regexp: log

docs           [Status: 200, Size: 920, Words: 53, Lines: 15]
external      [Status: 200, Size: 898, Words: 53, Lines: 15]
config        [Status: 200, Size: 909, Words: 54, Lines: 15]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
:: Progress: [220546/220546] :: Job [1/1] :: 9431 req/sec :: Duration: [0

```

General Options

These are the general parameters of this tool, which revolves around its general working on web fuzzing.

1. Custom Auto Calibration

We know that the power of a computer or machine to automatically calibrate itself is known as auto-calibration. Calibration is the process of providing a measuring instrument with the information it requires to understand the context in which it will be used. When gathering data, calibrating a computer ensures its accuracy.

We can customize this feature according to our need with the help of [-acc] parameter. Which can't be used without [-ac] parameter for its customization.

```

ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -acc -ac -fl 26 -ac -fs 2929 -ac
-fw 54

```

```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -acc -ac -fl 26 -ac -fs 2929 -ac -fw 54

v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist     : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration  : true
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
:: Filter      : Response size: 2929
:: Filter      : Response words: 54
:: Filter      : Response lines: 26

docs      [Status: 200, Size: 920, Words: 53, Lines: 15]
index     [Status: 302, Size: 0, Words: 1, Lines: 1]
security  [Status: 302, Size: 0, Words: 1, Lines: 1]
external  [Status: 200, Size: 898, Words: 53, Lines: 15]
logout    [Status: 302, Size: 0, Words: 1, Lines: 1]
about     [Status: 302, Size: 0, Words: 1, Lines: 1]
setup     [Status: 200, Size: 3549, Words: 182, Lines: 81]
login     [Status: 200, Size: 1289, Words: 83, Lines: 66]
instructions [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo   [Status: 302, Size: 0, Words: 1, Lines: 1]
:: Progress: [220546/220546] :: Job [1/1] :: 8270 req/sec :: Duration: [0:00:19] :: Errors: 0 ::

```

2. Color

Sometimes separation of colour creates extra attention to all details having in results. This [-c] parameter helps to create colour separation.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -c
```

```
(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -c
```



v1.2.1

```
:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
```

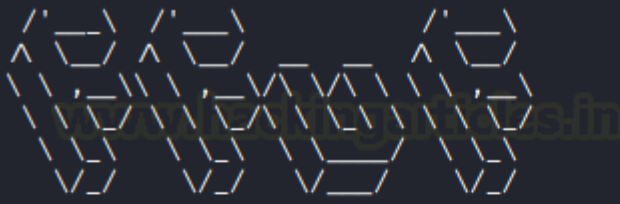
index	[Status: 302, Size: 0, Words: 1, Lines: 1]
login	[Status: 200, Size: 1289, Words: 83, Lines: 66]
about	[Status: 302, Size: 0, Words: 1, Lines: 1]
external	[Status: 200, Size: 898, Words: 53, Lines: 15]
logout	[Status: 302, Size: 0, Words: 1, Lines: 1]
security	[Status: 302, Size: 0, Words: 1, Lines: 1]
config	[Status: 200, Size: 909, Words: 54, Lines: 15]
setup	[Status: 200, Size: 3549, Words: 182, Lines: 81]
vulnerabilities	[Status: 200, Size: 2929, Words: 186, Lines: 26]
docs	[Status: 200, Size: 920, Words: 53, Lines: 15]
instructions	[Status: 302, Size: 0, Words: 1, Lines: 1]
	[Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo	[Status: 302, Size: 0, Words: 1, Lines: 1]

3. Maxtime For Task

If you want to fuzz for a limited amount of time then you can choose [-maxtime] parameter. Follow the command to provide a timeslot.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -maxtime 5
```

```
(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -maxtime 5
```



v1.2.1

```
:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
```

```
docs           [Status: 200, Size: 920, Words: 53, Lines: 15]
external       [Status: 200, Size: 898, Words: 53, Lines: 15]
config         [Status: 200, Size: 909, Words: 54, Lines: 15]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
instructions   [Status: 302, Size: 0, Words: 1, Lines: 1]
setup          [Status: 200, Size: 3549, Words: 182, Lines: 81]
index          [Status: 302, Size: 0, Words: 1, Lines: 1]
about          [Status: 302, Size: 0, Words: 1, Lines: 1]
security       [Status: 302, Size: 0, Words: 1, Lines: 1]
login          [Status: 200, Size: 1289, Words: 83, Lines: 66]
logout         [Status: 302, Size: 0, Words: 1, Lines: 1]
               [Status: 302, Size: 0, Words: 1, Lines: 1]
[WARN] Maximum running time for entire process reached, exiting.
```

4. Maxtime For Job

With the help of [-maxtime-job] parameter, we can put a time limit for a particular job. By using this command, we are trying to limit the time per job or request execution.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -maxtime-job 2
```

```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -maxtime-job 2

```



```

v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

docs      [Status: 200, Size: 920, Words: 53, Lines: 15]
index     [Status: 302, Size: 0, Words: 1, Lines: 1]
login     [Status: 200, Size: 1289, Words: 83, Lines: 66]
security  [Status: 302, Size: 0, Words: 1, Lines: 1]
external  [Status: 200, Size: 898, Words: 53, Lines: 15]
logout    [Status: 302, Size: 0, Words: 1, Lines: 1]
config    [Status: 200, Size: 909, Words: 54, Lines: 15]
setup     [Status: 200, Size: 3549, Words: 182, Lines: 81]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
about     [Status: 302, Size: 0, Words: 1, Lines: 1]
instructions [Status: 302, Size: 0, Words: 1, Lines: 1]
[WARN] Maximum running time for this job reached, continuing with next job if one exists.

```

5. Delay

If we create a particular delay in each request offered by the attack. Through this feature, a request has a better opportunity to get better results. The [-p] parameter help us to achieve delay in those requests.

```

ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -p 1

```

```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -p 1

```

v1.2.1

```

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Delay       : 1.00 seconds
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

```

```

index      [Status: 302, Size: 0, Words: 1, Lines: 1]
security   [Status: 302, Size: 0, Words: 1, Lines: 1]
docs       [Status: 200, Size: 920, Words: 53, Lines: 15]
about      [Status: 302, Size: 0, Words: 1, Lines: 1]
login      [Status: 200, Size: 1289, Words: 83, Lines: 66]

```

6. Request Rate

We can create a separate request rate for each of our attack with the help of the [-rate] parameter. Through this parameter, we create our request per second as per our attack desired.

```

ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -rate 500

```

```

(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -rate 500

v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist     : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

docs      [Status: 200, Size: 920, Words: 53, Lines: 15]
security  [Status: 302, Size: 0, Words: 1, Lines: 1]
index     [Status: 302, Size: 0, Words: 1, Lines: 1]
about     [Status: 302, Size: 0, Words: 1, Lines: 1]
login     [Status: 200, Size: 1289, Words: 83, Lines: 66]
external  [Status: 200, Size: 898, Words: 53, Lines: 15]
logout    [Status: 302, Size: 0, Words: 1, Lines: 1]
config    [Status: 200, Size: 909, Words: 54, Lines: 15]
setup     [Status: 200, Size: 3549, Words: 182, Lines: 81]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
instructions [Status: 302, Size: 0, Words: 1, Lines: 1]
:: Progress: [6143/220546] :: Job [1/1] :: 500 req/sec :: Duration: [0:00:28] :: Errors: 0 ::

```

7. Error Functions

There are three parameters that support the Error function. The first parameter is [-se], which is a spurious error. It states that the following request is genuine or not. The second parameter is [-sf], it will stop our attack when more than 95% of requests occurred as an error. The third and final parameter is [-sa], which is a combination of both the error parameter.

In our scenario, we are using [-se] parameter where it will stop our attack when our request is not real.

```

ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -rate 500

```



```

(root@kali)~# ffuf -u http://ignitetechnologies.in/W2/W1/ -w dict.txt:W1 -w dns_dict.txt:W2 -se

```



```

v1.2.1

:: Method      : GET
:: URL         : http://ignitetechnologies.in/W2/W1/
:: Wordlist    : W1: dict.txt
:: Wordlist    : W2: dns_dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

[WARN] Receiving spurious errors, exiting.

```

8. Verbose Mode

As we all know, the verbose mode is a feature used in many computers operating systems and programming languages that provide extra information on what the computer is doing and what drivers and applications it is loading at initialization. In programming, it produces accurate output for debugging purposes, making it easy to debug a program. There is a parameter called [-v] parameter.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -v
```

```

(root@kali)~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -v

```



v1.2.1

```

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

```

```

[Status: 200, Size: 920, Words: 53, Lines: 15]
| URL | http://192.168.1.12/dvwa/docs/
* FUZZ: docs

```

```

[Status: 200, Size: 898, Words: 53, Lines: 15]
| URL | http://192.168.1.12/dvwa/external/
* FUZZ: external

```

```

[Status: 200, Size: 909, Words: 54, Lines: 15]
| URL | http://192.168.1.12/dvwa/config/
* FUZZ: config

```

```

[Status: 200, Size: 2929, Words: 186, Lines: 26]
| URL | http://192.168.1.12/dvwa/vulnerabilities/
* FUZZ: vulnerabilities

```

```

[Status: 302, Size: 0, Words: 1, Lines: 1]
| URL | http://192.168.1.12/dvwa/security/
→ | login.php
* FUZZ: security

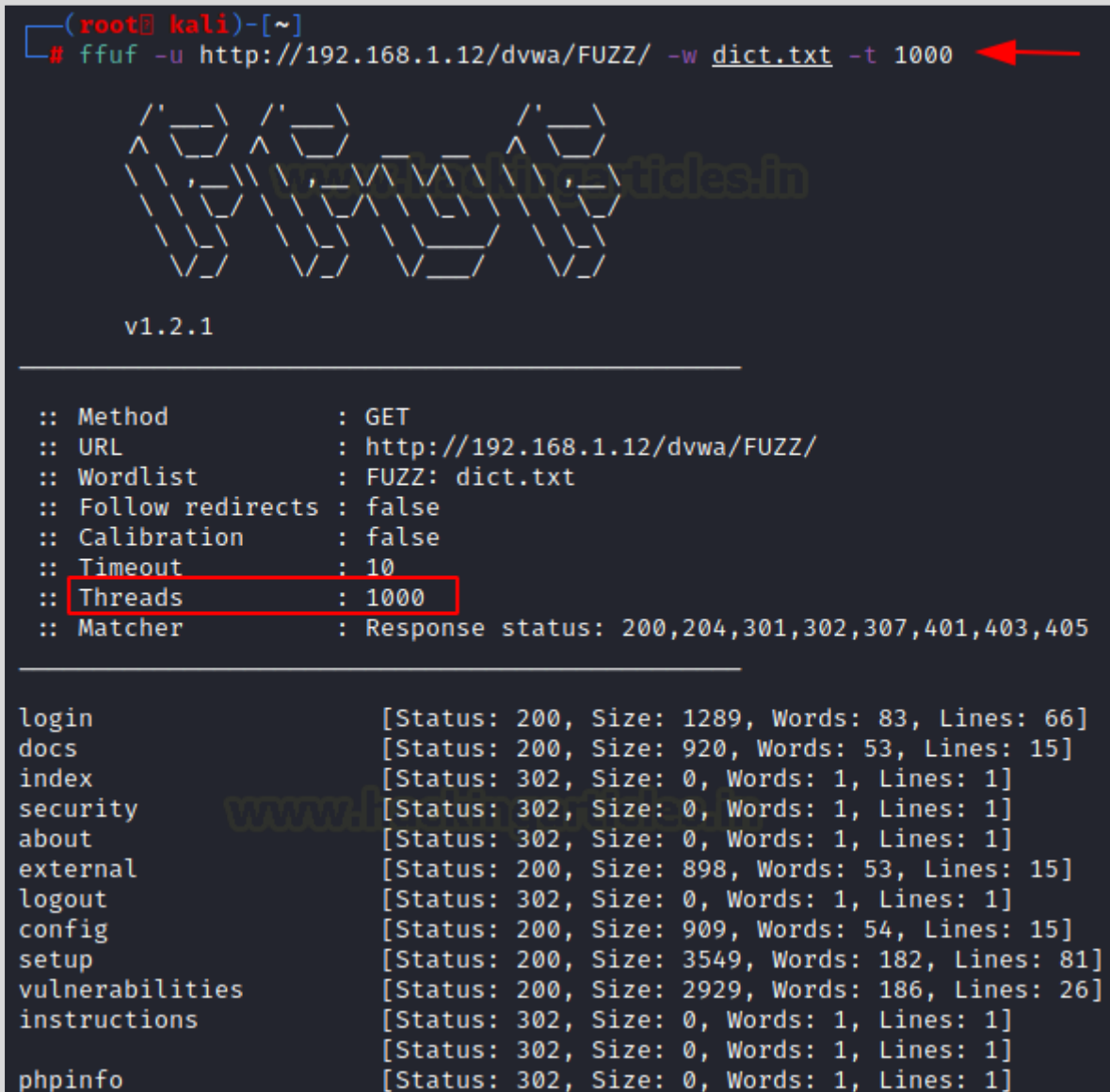
```

9. Threads

The [-t] parameter is used to speed up or slow down a process. By default, it is set on 40. if we want to pace up the process, we need to increase its number, vice versa to slow down process.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -v
```

```
(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -t 1000
```



```
v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 1000
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

login      [Status: 200, Size: 1289, Words: 83, Lines: 66]
docs       [Status: 200, Size: 920, Words: 53, Lines: 15]
index      [Status: 302, Size: 0, Words: 1, Lines: 1]
security   [Status: 302, Size: 0, Words: 1, Lines: 1]
about      [Status: 302, Size: 0, Words: 1, Lines: 1]
external   [Status: 200, Size: 898, Words: 53, Lines: 15]
logout     [Status: 302, Size: 0, Words: 1, Lines: 1]
config     [Status: 200, Size: 909, Words: 54, Lines: 15]
setup      [Status: 200, Size: 3549, Words: 182, Lines: 81]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
instructions [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo    [Status: 302, Size: 0, Words: 1, Lines: 1]
```

Output Options

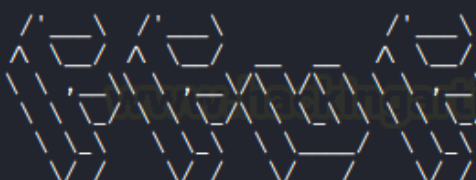
We save the performance of our attacks for the purposes of record-keeping, improved readability, and potential references. We use [-o] parameter to save our output, but we need to specify its format with [-of] parameter together.

1. Output Format in HTML

We use [-of] parameter and this defining with an HTML format. By using the command, we can create our report in html.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -o file.html -of html
```

```
(root@kali)~# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -o file.html -of html
```



```
v1.2.1
```

```
:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Output file : file.html
:: File format : html
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405
```

```
docs      [Status: 200, Size: 920, Words: 53, Lines: 15]
index     [Status: 302, Size: 0, Words: 1, Lines: 1]
login     [Status: 200, Size: 1289, Words: 83, Lines: 66]
about     [Status: 302, Size: 0, Words: 1, Lines: 1]
external  [Status: 200, Size: 898, Words: 53, Lines: 15]
logout    [Status: 302, Size: 0, Words: 1, Lines: 1]
security  [Status: 302, Size: 0, Words: 1, Lines: 1]
config    [Status: 200, Size: 909, Words: 54, Lines: 15]
setup     [Status: 200, Size: 3549, Words: 182, Lines: 81]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
instructions [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo   [Status: 302, Size: 0, Words: 1, Lines: 1]
:: Progress: [220546/220546] :: Job [1/1] :: 12036 req/sec :: Duration: [0:00:20]
```

Now after completion of this attack, we need to check our output file is up to that mark or not. As we can see that our file is successfully created.

file:///root/file.html

FFUF Report

ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -o file.html -of html

2021-03-19T14:33:03-04:00

Status	FUZZ	URL	Redirect location	Position	Length	Words	Lines
200	docs	http://192.168.1.12/dvwa/docs/		76	920	53	15
302	index	http://192.168.1.12/dvwa/index/	login.php	1	0	1	1
200	login	http://192.168.1.12/dvwa/login/		39	1289	83	66
302	about	http://192.168.1.12/dvwa/about/	login.php	12	0	1	1
200	external	http://192.168.1.12/dvwa/external/		1066	898	53	15
302	logout	http://192.168.1.12/dvwa/logout/	login.php	1211	0	1	1
302	security	http://192.168.1.12/dvwa/security/	login.php	57	0	1	1
200	config	http://192.168.1.12/dvwa/config/		1476	909	54	15
200	setup	http://192.168.1.12/dvwa/setup/		1884	3549	182	81
200	vulnerabilities	http://192.168.1.12/dvwa/vulnerabilities/		2794	2929	186	26
302	instructions	http://192.168.1.12/dvwa/instructions/	login.php	5107	0	1	1
302		http://192.168.1.12/dvwa/	login.php	45226	0	1	1
302	phpinfo	http://192.168.1.12/dvwa/phpinfo/	login.php	75572	0	1	1

2. Output Format in CSV

Similarly, we just need to csv format along with [-of] parameter. Where csv is a comma-separated values, which file allows you to store data in a tabular format.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -o file.csv -of csv
```

```

(root@ kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -o file.csv -of csv

```



 www.hackingarticles.in

v1.2.1

```

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Output file  : file.csv
:: File format  : csv
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

```

```

docs           [Status: 200, Size: 920, Words: 53, Lines: 15]
security       [Status: 302, Size: 0, Words: 1, Lines: 1]
external       [Status: 200, Size: 898, Words: 53, Lines: 15]
about          [Status: 302, Size: 0, Words: 1, Lines: 1]
logout         [Status: 302, Size: 0, Words: 1, Lines: 1]
config         [Status: 200, Size: 909, Words: 54, Lines: 15]
setup          [Status: 200, Size: 3549, Words: 182, Lines: 81]
index          [Status: 302, Size: 0, Words: 1, Lines: 1]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
login          [Status: 200, Size: 1289, Words: 83, Lines: 66]
instructions   [Status: 302, Size: 0, Words: 1, Lines: 1]
               [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo        [Status: 302, Size: 0, Words: 1, Lines: 1]
:: Progress: [220546/220546] :: Job [1/1] :: 9287 req/sec :: Duration: [0:00:20]

```

Now after completion of this attack, we need to check our output file is up to that mark or not. As we can see that our file is successfully created.

	A	B	C	D	E	F	G	H
1	FUZZ	url	redirectlocation	position	status_code	content_length	content_words	content_lines
2	docs	http://192.168.1.12/dvwa/docs/		76	200	920	53	15
3	security	http://192.168.1.12/dvwa/security/	login.php	57	302	0	1	1
4	external	http://192.168.1.12/dvwa/external/		1066	200	898	53	15
5	about	http://192.168.1.12/dvwa/about/	login.php	12	302	0	1	1
6	logout	http://192.168.1.12/dvwa/logout/	login.php	1211	302	0	1	1
7	config	http://192.168.1.12/dvwa/config/		1476	200	909	54	15
8	setup	http://192.168.1.12/dvwa/setup/		1884	200	3549	182	81
9	index	http://192.168.1.12/dvwa/index/	login.php	1	302	0	1	1
10	vulnerabilities	http://192.168.1.12/dvwa/vulnerabilities/		2794	200	2929	186	26
11	login	http://192.168.1.12/dvwa/login/		39	200	1289	83	66
12	instructions	http://192.168.1.12/dvwa/instructions/	login.php	5107	302	0	1	1
13		http://192.168.1.12/dvwa/	login.php	45226	302	0	1	1
14	phpinfo	http://192.168.1.12/dvwa/phpinfo/	login.php	75572	302	0	1	1
15								

3. All Output Format:

Similarly, if we want all output format at once just use [-of all] parameter. Like json, ejson, html, md, csv, ecsv. Follow this command to generate all reports at once.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -o output/file -of all
```

Now after completion of this attack, we need to check our output files is up to that mark or not. As we can see that our all files are successfully created.


```

(root@kali)~# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -o output/file -of all
v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Output file  : output/file.{json,ejson,html,md,csv,ecsv}
:: File format : all
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

login      [Status: 200, Size: 1289, Words: 83, Lines: 66]
index      [Status: 302, Size: 0, Words: 1, Lines: 1]
about      [Status: 302, Size: 0, Words: 1, Lines: 1]
external   [Status: 200, Size: 898, Words: 53, Lines: 15]
logout     [Status: 302, Size: 0, Words: 1, Lines: 1]
security   [Status: 302, Size: 0, Words: 1, Lines: 1]
config     [Status: 200, Size: 909, Words: 54, Lines: 15]
setup      [Status: 200, Size: 3549, Words: 182, Lines: 81]
vulnerabilities [Status: 200, Size: 2929, Words: 186, Lines: 26]
docs       [Status: 200, Size: 920, Words: 53, Lines: 15]
instructions [Status: 302, Size: 0, Words: 1, Lines: 1]
phpinfo    [Status: 302, Size: 0, Words: 1, Lines: 1]
:: Progress: [220546/220546] :: Job [1/1] :: 7595 req/sec :: Duration: [0:00:19] ::

(root@kali)~# cd output

(root@kali)~/output# ls
file.csv file.ecsv file.ejson file.html file.json file.md

```

HTTP Options

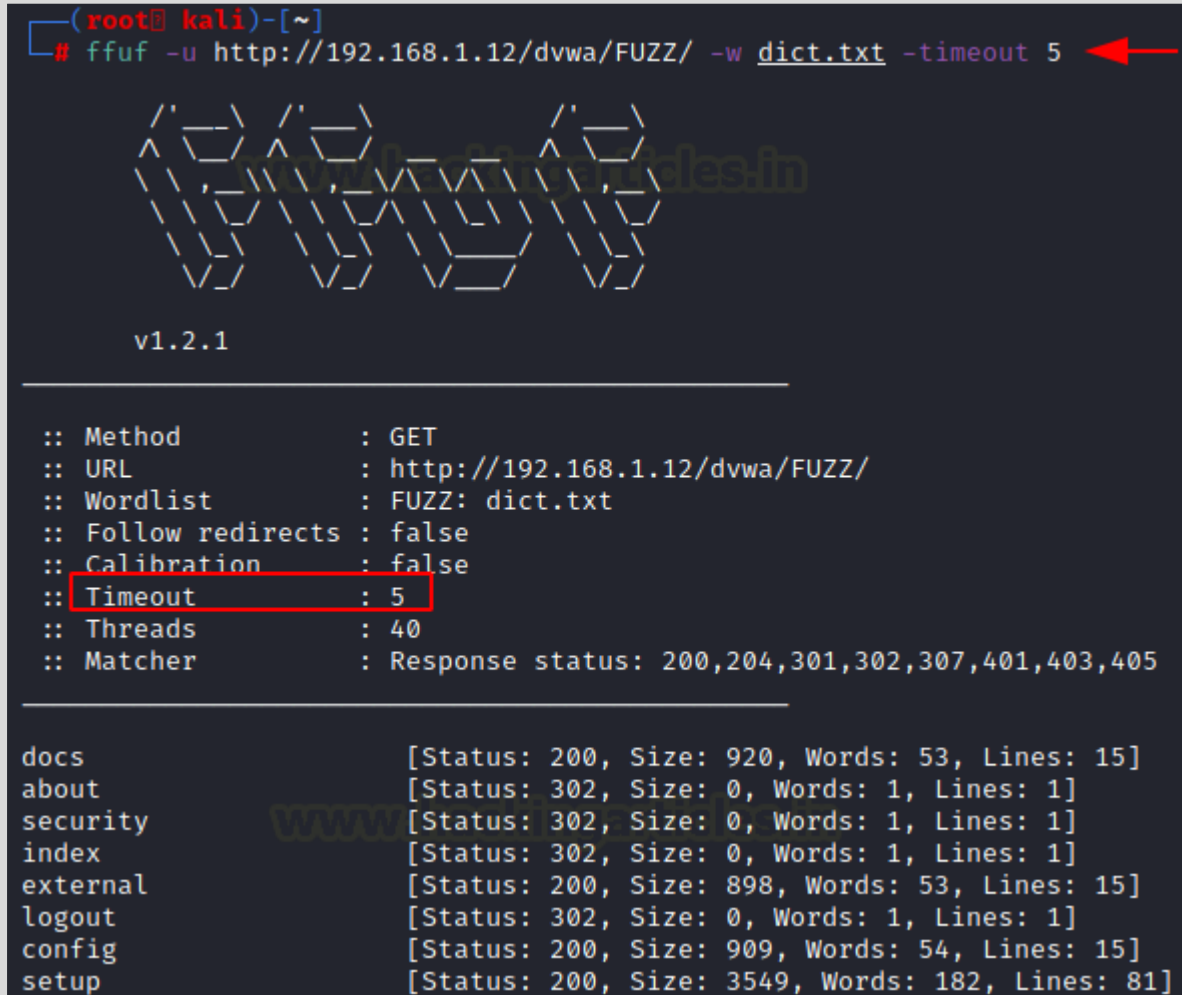
The options move around HTTP options, sometimes it required the details to run web fuzzing Like HTTP request, Cookie, HTTP header, etc.

1. Timeout:

Timeout act as a deadline for the event. The [-timeout] parameter help of established this feature with ease, follow this command to run this parameter.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -timeout 5
```

```
(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -timeout 5
```



```
v1.2.1
```

```
:: Method          : GET
:: URL             : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist        : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration     : false
:: Timeout         : 5
:: Threads         : 40
:: Matcher         : Response status: 200,204,301,302,307,401,403,405
```

```
docs           [Status: 200, Size: 920, Words: 53, Lines: 15]
about          [Status: 302, Size: 0, Words: 1, Lines: 1]
security       [Status: 302, Size: 0, Words: 1, Lines: 1]
index          [Status: 302, Size: 0, Words: 1, Lines: 1]
external       [Status: 200, Size: 898, Words: 53, Lines: 15]
logout         [Status: 302, Size: 0, Words: 1, Lines: 1]
config         [Status: 200, Size: 909, Words: 54, Lines: 15]
setup          [Status: 200, Size: 3549, Words: 182, Lines: 81]
```

2. Host Header

If we want to perform fuzzing on subdomain, we can use [-H] parameter along with a domain name wordlist as given below in the command.

```
ffuf -u https://google.com -w dns_dict.txt
-mc 200 -H "HOST: FUZZ.google.com"
```

```
(root@kali)-[~]
# ffuf -u https://google.com -w dns_dict.txt -mc 200 -H "HOST: FUZZ.google.com"

v1.2.1

:: Method      : GET
:: URL         : https://google.com
:: Wordlist     : FUZZ: dns_dict.txt
:: Header      : Host: FUZZ.google.com
:: Follow redirects : false
:: Calibration : false
:: Timeout      : 10
:: Threads      : 40
:: Matcher      : Response status: 200

www      [Status: 200, Size: 15209, Words: 359, Lines: 14]
images   [Status: 200, Size: 12685, Words: 305, Lines: 13]
support  [Status: 200, Size: 680332, Words: 17873, Lines: 3412]
```

3. Recursion

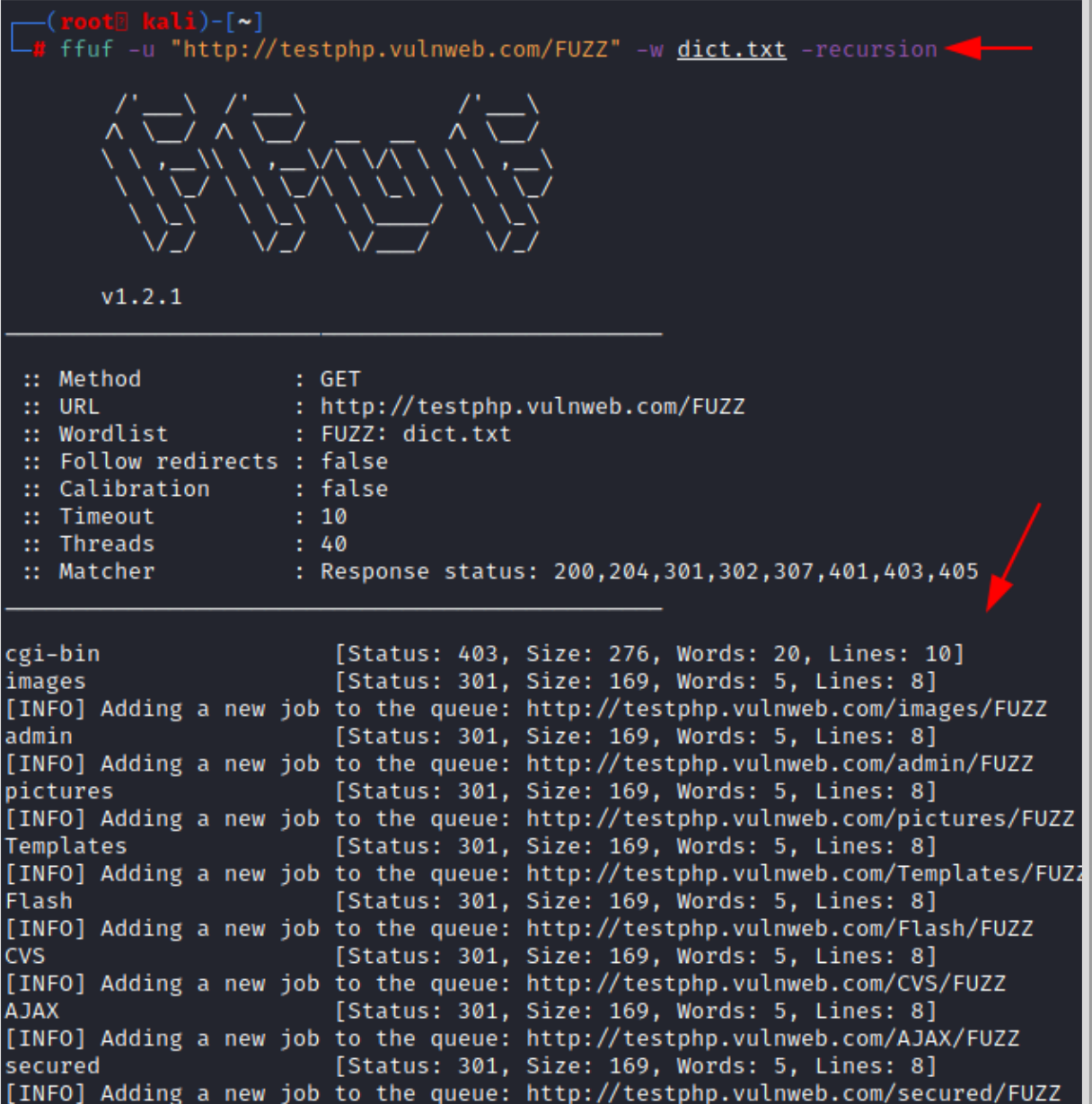
Recursion is the mechanism of repeating objects in a self-similar manner, as we all know. If a program requires you to access a function within another function, this is referred to as a recursive call of the function. By using `[-recursion]` parameter, we can achieve this functionality in our attacks.

```
ffuf -u https://google.com -w dns_dict.txt
-recursion
```

```

(root@kali)-[~]
# ffuf -u "http://testphp.vulnweb.com/FUZZ" -w dict.txt -recursion

```



```

v1.2.1

:: Method      : GET
:: URL         : http://testphp.vulnweb.com/FUZZ
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

cgi-bin      [Status: 403, Size: 276, Words: 20, Lines: 10]
images       [Status: 301, Size: 169, Words: 5, Lines: 8]
[INFO] Adding a new job to the queue: http://testphp.vulnweb.com/images/FUZZ
admin        [Status: 301, Size: 169, Words: 5, Lines: 8]
[INFO] Adding a new job to the queue: http://testphp.vulnweb.com/admin/FUZZ
pictures     [Status: 301, Size: 169, Words: 5, Lines: 8]
[INFO] Adding a new job to the queue: http://testphp.vulnweb.com/pictures/FUZZ
Templates   [Status: 301, Size: 169, Words: 5, Lines: 8]
[INFO] Adding a new job to the queue: http://testphp.vulnweb.com/Templates/FUZZ
Flash       [Status: 301, Size: 169, Words: 5, Lines: 8]
[INFO] Adding a new job to the queue: http://testphp.vulnweb.com/Flash/FUZZ
CVS         [Status: 301, Size: 169, Words: 5, Lines: 8]
[INFO] Adding a new job to the queue: http://testphp.vulnweb.com/CVS/FUZZ
AJAX        [Status: 301, Size: 169, Words: 5, Lines: 8]
[INFO] Adding a new job to the queue: http://testphp.vulnweb.com/AJAX/FUZZ
secured     [Status: 301, Size: 169, Words: 5, Lines: 8]
[INFO] Adding a new job to the queue: http://testphp.vulnweb.com/secured/FUZZ

```

4. Attack with Cookie

Sometimes web fuzzing does not show the result on authenticated site without authentication. There is a [-b] parameter through which we can achieve your goal by providing a session cookie.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -b
```

```
(root@kali)~[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -b "PHPSESSID:7aaaa6d8edcf7cd2ea4e3853ebb8bde"

v1.2.1

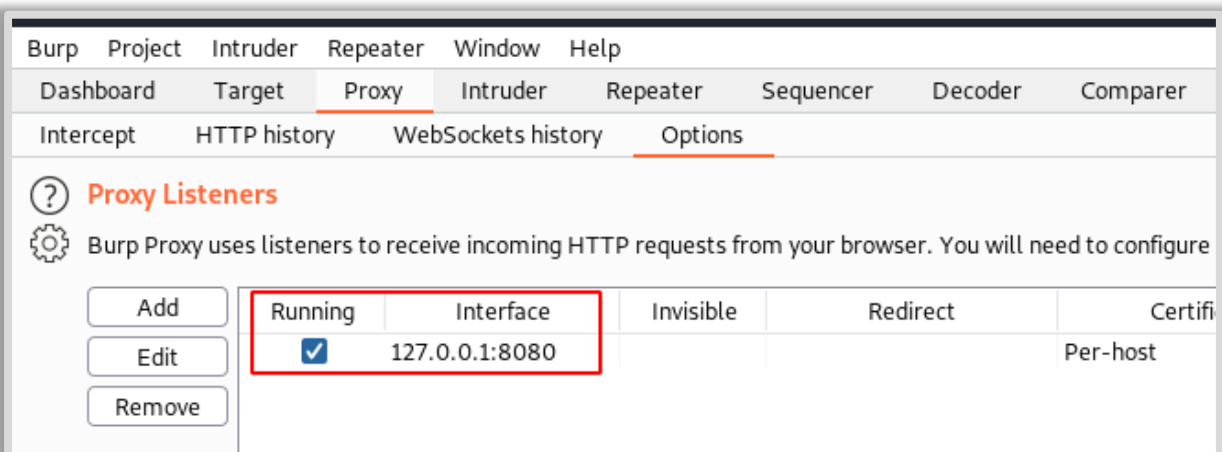
:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist     : FUZZ: dict.txt
:: Header      : Cookie: PHPSESSID:7aaaa6d8edcf7cd2ea4e3853ebb8bde
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403,405

docs      [Status: 200, Size: 920, Words: 53, Lines: 15]
external  [Status: 200, Size: 898, Words: 53, Lines: 15]
config    [Status: 200, Size: 909, Words: 54, Lines: 15]
```

5. Replay-Proxy

As you might be aware, there are speed restrictions when using the Intruder function in the free version of the Burp suite (Community Edition). The Intruder attack has been severely slowed, with each order slowing the attack even further.

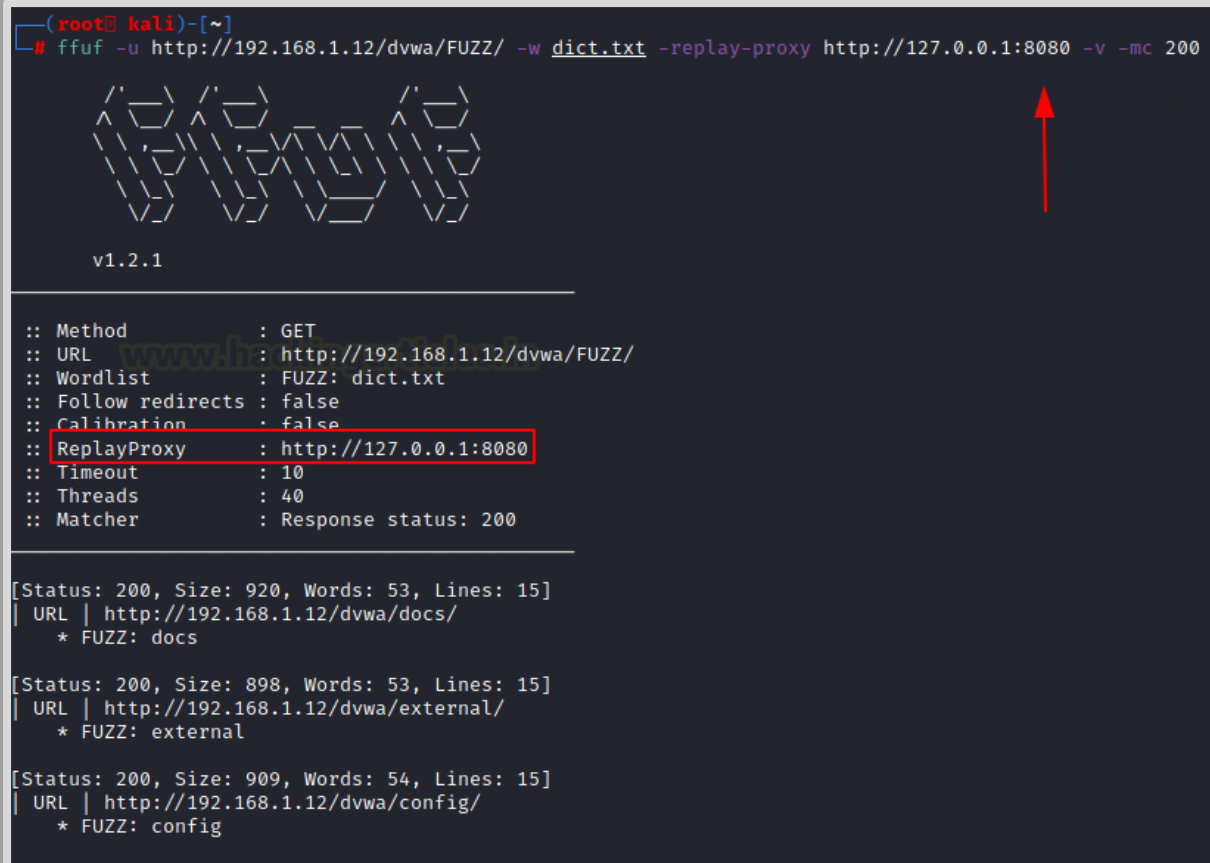
In our case we are using Burp suite proxy to get results for evaluation in it. First, we have to establish a localhost proxy on port 8080.



Now use [-replay-proxy] parameter, which helps us to derive our local host proxy which we established in the previous step on port 8080 along with our attack.

```
ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w
dict.txt -replay-proxy http://127.0.0.1:8080
-v -mc 200
```

```
(root@kali)-[~]
# ffuf -u http://192.168.1.12/dvwa/FUZZ/ -w dict.txt -replay-proxy http://127.0.0.1:8080 -v -mc 200
```



```

v1.2.1

:: Method      : GET
:: URL         : http://192.168.1.12/dvwa/FUZZ/
:: Wordlist    : FUZZ: dict.txt
:: Follow redirects : false
:: Calibration : false
:: ReplayProxy  : http://127.0.0.1:8080
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200

[Status: 200, Size: 920, Words: 53, Lines: 15]
| URL | http://192.168.1.12/dvwa/docs/
* FUZZ: docs

[Status: 200, Size: 898, Words: 53, Lines: 15]
| URL | http://192.168.1.12/dvwa/external/
* FUZZ: external

[Status: 200, Size: 909, Words: 54, Lines: 15]
| URL | http://192.168.1.12/dvwa/config/
* FUZZ: config

```

This attack will show our results on two platforms. The first platform on the kali terminal and the second on the Burp suite HTTP history tab. Through these various techniques, we can better understand our target and our attack results.

Dashboard	Target	Proxy	Intruder	Repeater	Sequencer	Decoder
Intercept	HTTP history	WebSockets history		Options		
Filter: Hiding CSS, image and general binary content						
#	Host	Method	URL	Edited ▾	Status	Length
9	http://192.168.1.12	GET	/dvwa/docs/		200	1092
10	http://192.168.1.12	GET	/dvwa/external/		200	1070
11	http://192.168.1.12	GET	/dvwa/config/		200	1081
12	http://192.168.1.12	GET	/dvwa/setup/		200	3950
13	http://192.168.1.12	GET	/dvwa/login/		200	1690
14	http://192.168.1.12	GET	/dvwa/vulnera...		200	3102

Conclusion

The ffuf is often compared to tools like dirb or dirbuster, which, although accurate to certain extents, isn't a reasonable analogy. Although FFUF can be used to brute force files, its true strength lies in its simplicity, and a better comparative tool for FFUF would be anything like Burp Suite Intruder or Turbo Intruder.

References

- <https://www.hackingarticles.in/comprehensive-guide-on-ffuf/>

JOIN OUR TRAINING PROGRAMS

